

Test plan: live-testing option 3B with `webview2-proxy-probe`

Written 2026-09-23. Companion to `option-3b-per-process-proxy-design.md` — everything in that doc about `WEBVIEW2_ADDITIONAL_BROWSER_ARGUMENTS` was confirmed from Microsoft's own documentation and Q&A threads, never live-tested. This plan runs the actual mechanism against real Word, Excel, PowerPoint, and Outlook (new and classic), using the probe tool in `ai-protect/ai-gateway/webview2-proxy-probe`.

What this test answers

For each app: does setting `WEBVIEW2_ADDITIONAL_BROWSER_ARGUMENTS` (user-scoped, in `HKCU\Environment`) actually cause the Copilot chat pane's network traffic to route through a local listener on this port — with zero cooperation from Office, no code change, nothing but a registry value?

A **pass** for an app closes Reason 1 for that app's Copilot traffic, for free. A **fail** means that app's Copilot pane either isn't WebView2-hosted after all, or doesn't route its network calls through the WebView2/Chromium layer the env var reaches (i.e. the connection is made by Office's native code, with WebView2 only used to *render* the response) — in which case option 4 is the fallback for that app specifically, not for 3B as a whole.

Prerequisites

- A Windows machine with Word, Excel, PowerPoint, and Outlook installed, signed into an account with Microsoft 365 Copilot access.
- Rust toolchain (`cargo build` — no admin rights needed; `HKCU\Environment` is a per-user key, not `HKLM`).
- Both classic Outlook and new Outlook available to test independently if possible (the design doc's confidence table treats them as separate cases) — if the machine only has one, note which in the results.

Build

```
cd ai-protect
cargo build -p webview2-proxy-probe --release
```

Procedure

Run one app at a time — the log doesn't distinguish which process a connection came from, so mixing apps in one run would make the results ambiguous. Repeat steps 2-6 per app.

1. **Confirm clean state:** `powershell webview2-proxy-probe.exe status` Should report the value is not set (or `revert` first if a prior run left it behind).
2. **Set the redirect:** `powershell webview2-proxy-probe.exe set --port 58080`
3. **Sign out and back in.** Not optional — see the tool's own printed warning. A process launched by Explorer before it has refreshed its own environment block won't inherit the new value, and that's the common case for how Office actually gets launched (Start menu, taskbar, a file association) — closing and reopening the app alone is not equivalent to a fresh sign-in for this purpose.
4. **Start the listener**, in its own terminal, kept open for the duration of this app's test: `powershell webview2-proxy-probe.exe listen --port 58080 --log-file probe-<app>.log` Replace `<app>` per test (e.g. `probe-word.log`) so each app's run is captured separately.
5. **Open the app, open the Copilot chat pane, send a real prompt** (e.g. "summarize this document" or any prompt that requires a real model response, not just UI navigation — the goal is to provoke the actual Chathub network call, not just render the pane).
6. **Check the listener's output and log file** for any request whose host is `substrate.office.com`, `copilot.cloud.microsoft`, or anything else under `*.cloud.microsoft / *.office.com`. Record:
 - Did *any* connection arrive at all?
 - If yes: what host/method (`CONNECT host:443` for the real SignalR WebSocket upgrade is the strongest positive signal — a plain `GET` might just be a background telemetry call, not the Chathub connection itself)?
 - Did the Copilot pane actually still work (get a real response) while routed through the probe? This confirms the transparent-relay path didn't break anything, and rules out "it silently fell back to a direct connection because the proxy failed."
7. **Stop the listener** (Ctrl+C) before moving to the next app, so each app's log file is a clean, separate capture.

After all apps are tested:

8. **Revert:** `powershell webview2-proxy-probe.exe revert` Sign out and back in again to fully restore normal behavior before returning the machine to regular use.

Results table — fill in and feed back into the design doc

App	Connection observed?	Host/method seen	Copilot pane still worked?	Verdict
Word (WINWORD.EXE)				
Excel (EXCEL.EXE)				
PowerPoint (POWERPNT.EXE)				
Outlook — new				
Outlook — classic				

Verdict should be one of:

- **Confirmed** — a real Chathub-shaped connection arrived and the pane kept working. Update that app's row in the design doc's confidence table from "confirmed [tier]" to "live-tested, confirmed."
- **Refuted** — no connection arrived at all, or the pane broke while nothing relevant showed up in the log (native-code path, not Chromium-originated). Update the design doc to record this app needs option 4 instead.
- **Inconclusive** — something arrived but it's unclear whether it's the real Chathub connection (e.g. only a background telemetry GET , no CONNECT) — worth a follow-up pass with a more deliberate prompt, not a final answer either way.

After the run

Whatever the results, update option-3b-per-process-proxy-design.md 's confidence table and "Where this leaves 3B" section with the real, live outcome per app — this test plan's job is done once that doc reflects reality instead of documentation-only confidence.